❷ The username is assigned to the user.

❸ The email is assigned to the user.

❹ The user is saved to the data store by the UserManager.

In previous versions of the Identity UI, the user was created with its email and usernames in one line of code:

```
var user = new CityBreaksUser { UserName = Input.Email, Email = Input.Email };
```

The updated version released as part of .NET 6 is a lot more verbose. Creation is delegated to a private method named `CreateUser` that uses `Activator.CreateInstance` with error handling for some edge cases the scaffold developers couldn't foresee. You don't have to stick with this. You can replace the code with the simpler version instead. After all, this is your code now.

There are other differences from previous versions. The username and email properties are assigned via the `IUserStore.SetUserNameAsync` and `IUserEmailStore.SetEmailAsync` methods, respectively. These APIs provide a consistent way to get and set the username and email for the user. There is also a similar API for managing phone numbers: the `IUserPhoneNumberStore` interface. I don't go into any real detail on these interfaces in this book. It is enough to know that the default implementations simply assign the specified value (`Input.Email`) to the user's `UserName` and `Email` properties. All you need to do for the time being is assign incoming values to the user's new properties.

Listing 9.19 Assigning values to the Identity user's custom properties

```
if (ModelState.IsValid)
{
    var user = CreateUser();

    user.FirstName = Input.FirstName;      ❶
    user.LastName = Input.LastName;        ❶
    user.Address = Input.Address;          ❶
```